# DOM-Based Cross-Site Scripting (DOM XSS)

**Severity**: Medium — Context-dependent

## 1. About DOM-Based XSS

DOM-Based Cross-Site Scripting (DOM XSS) is a client-side vulnerability that occurs when an application processes untrusted user input through JavaScript and inserts it into the Document Object Model (DOM) without proper handling.

An attacker can exploit this weakness to execute malicious JavaScript in another user's browser under the application's origin.

Unlike traditional reflected XSS, DOM XSS can occur entirely within the browser without requiring the server to include the malicious input in its HTTP response.

## 2. Payload Used

The following payload was used to complete the DOM XSS challenge:
**<iframe src="javascript:alert(`xss`)">**

**Payload explanation:**
- <iframe> creates an embedded browsing context.
- src specifies the content or URL loaded inside the iframe.
- javascript: instructs the browser to execute JavaScript as a URL.
- alert('xss') displays a popup to demonstrate JavaScript execution.

**Note**: The intended protocol spelling is javascript:. If your notes say javascriDDDpt:, that is a typo or altered spelling, not the working protocol.

## 3. Severity and Security Impact

**Suggested severity:** Medium (context-dependent).
The actual severity depends on the application's functionality, user privileges, affected users, and the sensitivity of accessible information. If successfully exploited against another user, DOM XSS could allow an attacker to:

- Access sensitive information available to JavaScript within the page.
- Read authentication tokens or session-related data when they are accessible to JavaScript.
- Modify page content or display fraudulent login forms.
- Perform unauthorized actions using the victim's active session, depending on application protections.
- Access or manipulate data available through the application's client-side functionality.

**Important**: The alert popup confirms JavaScript execution in the lab. It does not independently prove that sensitive information was accessed or stolen.

## 4. Remediation

The following security measures can help prevent DOM-Based XSS:

**1. Use safe DOM methods**
Avoid inserting untrusted user input directly through dangerous methods such as innerHTML and

document.write().

**Prefer: element.textContent = userInput;**

This treats the supplied input as text instead of interpreting it as HTML.

**2. Sanitize untrusted HTML**

When an application genuinely needs to display user-supplied HTML, use a trusted HTML sanitizer such as DOMPurify with an appropriate security configuration.

**3. Validate URL protocols**

Restrict user-controlled URLs to approved protocols, such as https: and http:. Reject dangerous schemes such as javascript:.

**4. Implement Content Security Policy (CSP)**

Configure a restrictive CSP to reduce the impact of script injection. CSP is an additional security layer, not a replacement for fixing unsafe DOM operations.

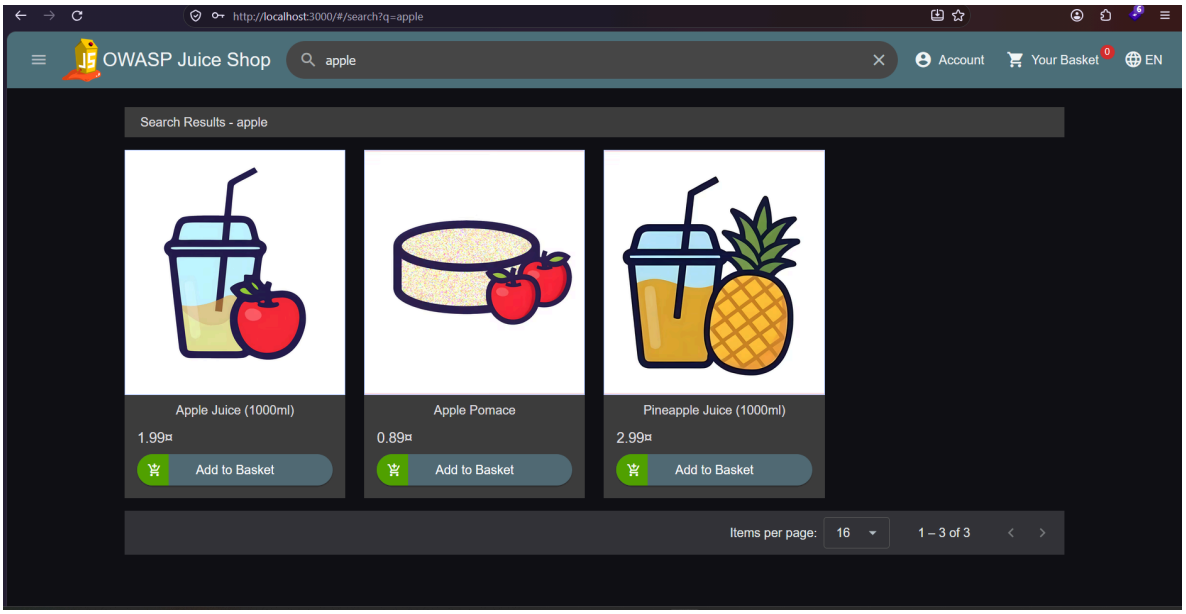
**5. Apply proper output encoding**

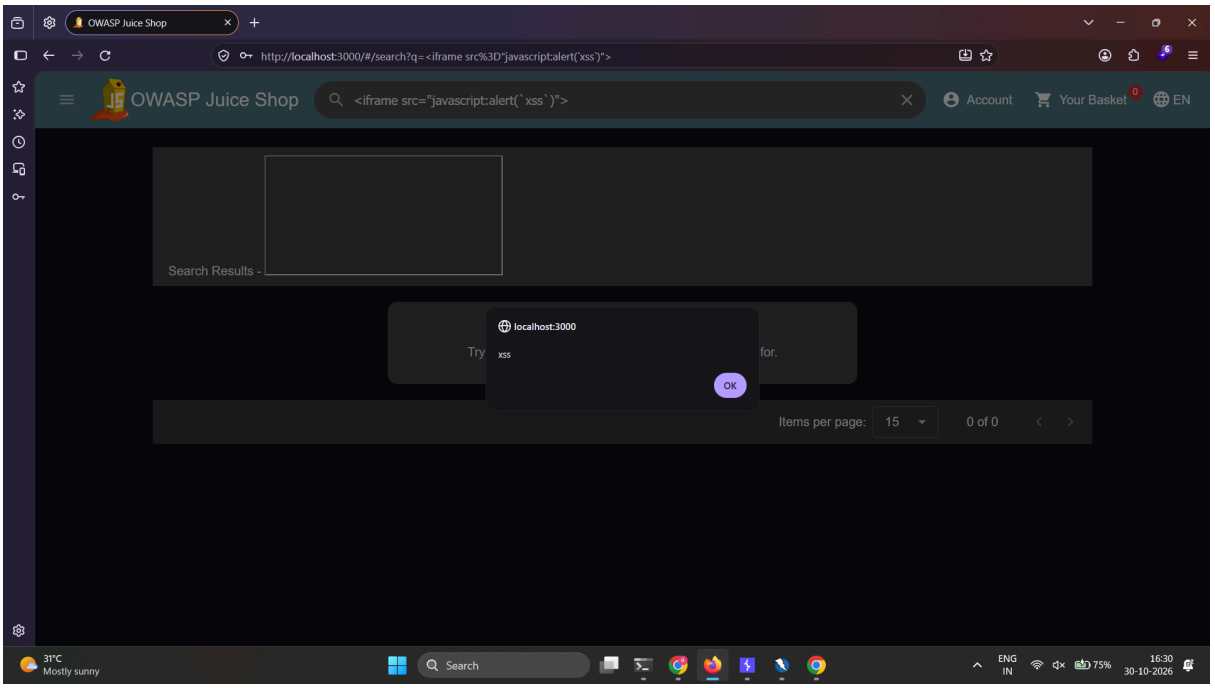
Encode untrusted data according to its output context to prevent it from being interpreted as executable code or HTML.
These recommendations follow the OWASP DOM-Based XSS Prevention guidelines.

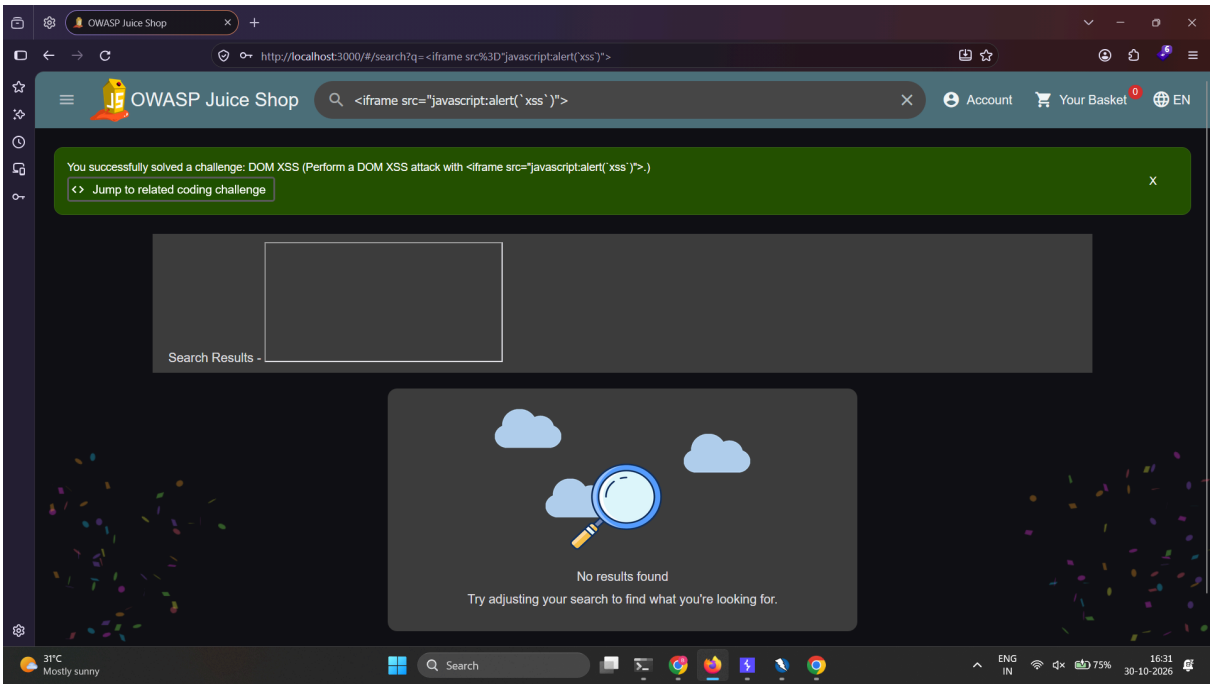
# 5. Result

Successfully completed the DOM-Based XSS challenge in OWASP Juice Shop by injecting an iframe containing a JavaScript URL. The browser executed the supplied JavaScript and displayed an alert popup, demonstrating the potential for client-side script execution.

This exercise helped me understand unsafe DOM manipulation, JavaScript URL execution, client-side security weaknesses, and XSS prevention techniques.






# References


OWASP Juice Shop — DOM XSS Challenge
OWASP — DOM-Based XSS Prevention Cheat Sheet
OWASP — Cross-Site Scripting Prevention Cheat Sheet


## DOM XSS: Identifying and Fixing the Vulnerability

**Concept:** This challenge demonstrates how unsafe handling of URL query parameters can introduce DOM-Based Cross-Site Scripting (XSS) vulnerabilities in an Angular application.

## 1. Vulnerability Identification (Find It)

The vulnerable code retrieves user input from the URL query parameter q:
let queryParam: **string = this.route.snapshot.queryParams.q;**
The vulnerability occurs at: **this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam);**

**Explanation**: The bypassSecurityTrustHtml() method tells Angular to trust the supplied content as safe HTML, bypassing its normal HTML sanitization. If untrusted input is later rendered as HTML, it may allow JavaScript injection.

## 2. Remediation (Fix It)

Selected Fix 4, which removes the unnecessary sanitization bypass.
Vulnerable code: **this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam);**
Fixed code: **this.searchValue = queryParam;**

**Explanation:** Since the search input is expected to contain plain text rather than HTML, there is no need to bypass Angular's built-in protections. The input should be handled as ordinary text.

## 3. Security Impact

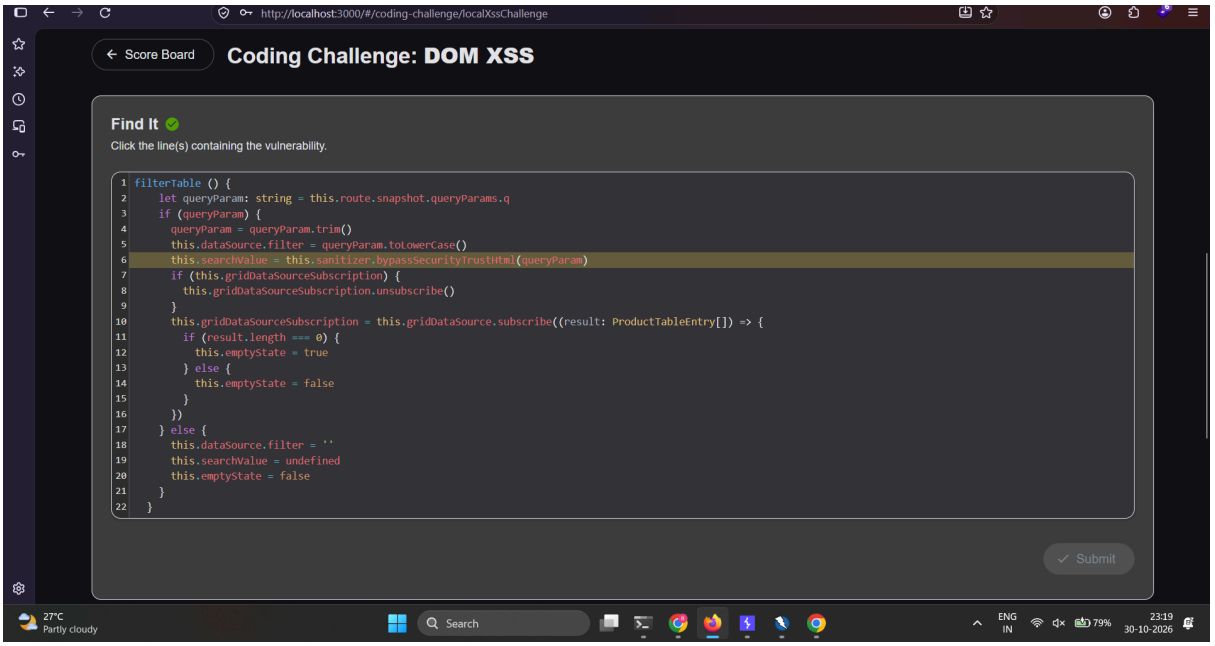
An attacker could potentially inject malicious HTML or JavaScript through URL parameters if the application renders the input unsafely. This could lead to unauthorized actions, manipulation of page content, or exposure of information accessible to JavaScript.

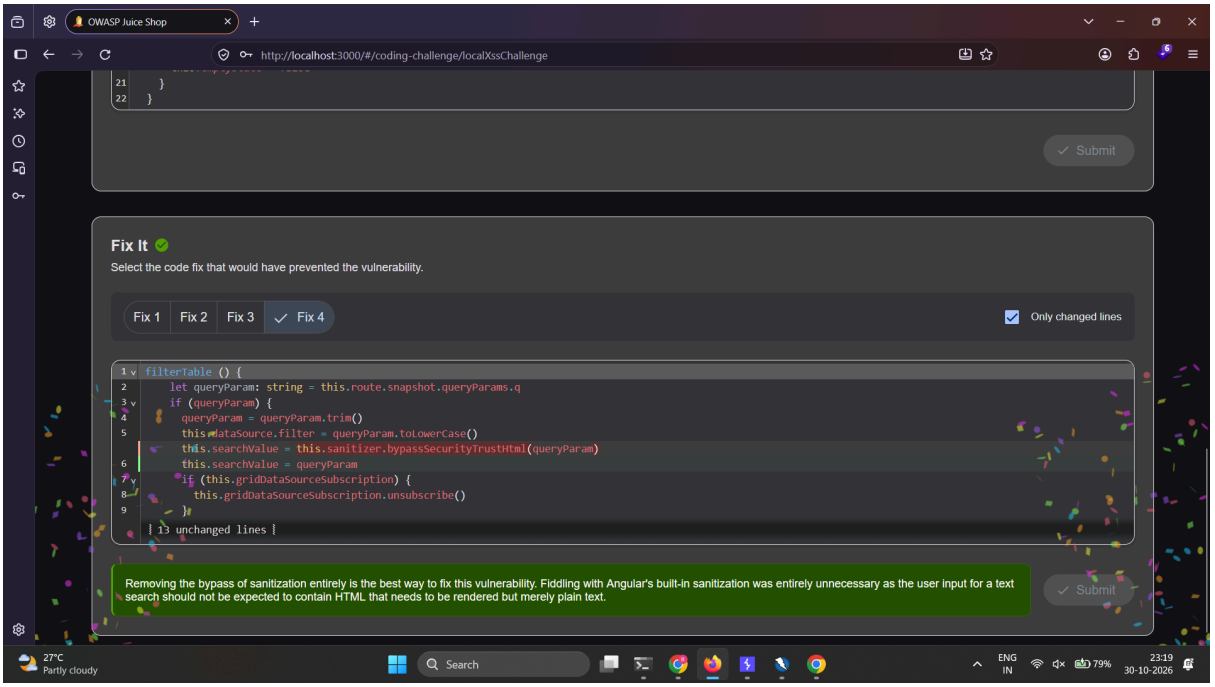
## 4. Result

Successfully completed both the Find It and Fix It sections of the OWASP Juice Shop DOM XSS coding challenge. Identified the unsafe use of bypassSecurityTrustHtml() and applied Fix 4 to remove the unnecessary sanitization bypass.

**Key Learning:** Never trust user-controlled input as safe HTML. Use Angular's standard sanitization and render plain-text input as text whenever HTML rendering is not required.

**Evidence:**






**Severity**: Medium (context-dependent). The coding challenge demonstrates an unsafe coding pattern; the real-world severity depends on whether the value reaches an executable HTML rendering context and what data or functionality is accessible.